



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
DISCIPLINA: ALGORITMOS E ESTRUTURAS DE DADOS I
DOCENTE: Dra. Rosana Cibely Batista Rego

Sistema de Prontuário Eletrônico em Linguagem C

Discentes: Maximiliano Gois Sena;
José Augusto de Souza Costa;
João Vitor Ribeiro Pacó

MOSSORÓ
ANO

1. Introdução

O trabalho desenvolvido consiste em um sistema de prontuário médico eletrônico implementado em linguagem C. O objetivo principal foi construir uma aplicação capaz de cadastrar pacientes, registrar consultas, pesquisar prontuários, editar dados, excluir consultas, listar informações armazenadas e manter os dados salvos em arquivos texto para que não fossem perdidos ao encerrar o programa.

A proposta também teve como objetivo aplicar conteúdos estudados na disciplina, principalmente tipos estruturados, ponteiros, alocação dinâmica de memória, listas encadeadas, TADs, arquivos e validação de entradas. Para isso, o programa foi dividido em módulos, evitando concentrar toda a lógica dentro da função main e facilitando a manutenção do código.

A estrutura de dados escolhida para armazenar pacientes e consultas foi a lista encadeada. Essa escolha foi feita porque a quantidade de pacientes e consultas não é fixa. Assim, o programa pode crescer durante a execução, alocando memória apenas quando um novo paciente ou uma nova consulta é cadastrada.

2. Desenvolvimento

2.1 Organização geral do projeto

O projeto foi organizado em arquivos separados. O arquivo main.c contém o menu principal e controla o fluxo inicial do programa. O TAD Paciente foi implementado nos arquivos paciente.h e paciente.c, enquanto o TAD Consulta foi implementado nos arquivos consulta.h e consulta.c. Além disso, foi criado o modulo validacao.h e validacao.c, responsável por concentrar as regras de leitura e validação dos dados informados pelo usuário.

A divisão em TADs foi utilizada para separar responsabilidades. O TAD Paciente cuida dos dados do paciente e da lista de pacientes. O TAD Consulta cuida das consultas associadas a cada paciente. Já o modulo de validação evita repetição de código, pois as mesmas regras de entrada são usadas em diferentes partes do sistema.

- main.c: apresenta o menu e chama as funções principais do sistema.
- paciente.h e paciente.c: definem e implementam o TAD Paciente.
- consulta.h e consulta.c: definem e implementam o TAD Consulta.
- validacao.h e validacao.c: reúnem as funções auxiliares de validação.
- pacientes.txt e consultas.txt: armazenam os dados persistentes em formato textual.

2.2 Estrutura de dados utilizada

Foi utilizada uma lista encadeada para armazenar os pacientes. Cada paciente possui campos como número do prontuário, nome, CPF, data de nascimento e telefone. Além disso, cada paciente possui um ponteiro para sua própria lista de consultas. Dessa forma, a organização do sistema permite que um paciente tenha nenhuma, uma ou várias consultas registradas.

A pilha não foi escolhida porque seu funcionamento é baseado no princípio LIFO, em que o último elemento inserido é o primeiro a ser removido. Esse comportamento não combina com um prontuário, pois o sistema precisa buscar, listar e editar qualquer paciente. A fila também

não foi escolhida como estrutura principal, pois trabalha com o princípio FIFO, mais adequado para ordem de atendimento. Como o sistema precisava permitir cadastro, busca, edição, exclusão e percorrimento dos dados, a lista encadeada foi a estrutura mais adequada.

2.3 TAD Paciente

O TAD Paciente concentra as funções relacionadas ao gerenciamento dos pacientes. A função `criarListaPacientes` retorna uma lista vazia, representada por `NULL`. A função `cadastrarPaciente` utiliza `malloc` para alocar dinamicamente um novo paciente, solicita os dados ao usuário por meio das funções de validação, associa uma lista vazia de consultas ao paciente e insere esse novo registro no início da lista encadeada.

A função `proximoNumeroProntuario` foi criada para gerar automaticamente o número do prontuário. Ela percorre a lista de pacientes, identifica o maior número já cadastrado e retorna o próximo valor disponível. Isso evita que o usuário precise informar manualmente o número do prontuário.

A função `buscarPacientePorCPF` percorre a lista e compara o CPF informado com o CPF de cada paciente. Ela é usada para localizar pacientes nas operações de busca, registro de consulta, exclusão de consulta e edição de prontuário. Já a função `buscarPacientePorCPFExceto` foi criada para a edição do CPF, pois permite verificar se existe outro paciente com o mesmo CPF, ignorando o próprio paciente que está sendo editado.

As funções `listarPacientes` e `mostrarPaciente` foram usadas para exibir informações. `listarPacientes` mostra todos os pacientes cadastrados e a quantidade de consultas de cada um. `mostrarPaciente` exibe os dados completos de um único paciente e também chama a listagem de consultas daquele paciente.

A função `editarProntuario` permite alterar nome, CPF, data de nascimento e telefone. Cada alteração utiliza as funções de validação apropriadas. Ao final, `salvarDados` é chamada para gravar as mudanças nos arquivos. A função `liberarListaPacientes` percorre todos os pacientes, libera primeiro as consultas associadas e depois libera a memória do paciente, evitando vazamento de memória.

2.4 TAD Consulta

O TAD Consulta foi criado para controlar as consultas de cada paciente. A função `criarListaConsultas` retorna `NULL`, indicando que um paciente ainda não possui consultas. A função `proximoCodigoConsulta` percorre a lista de consultas de um paciente e gera automaticamente o próximo código disponível para identificar cada atendimento.

A função `cadastrarConsulta` é responsável por registrar um novo atendimento. Ela solicita data, horário, nome do médico, sintomas, diagnóstico e prescrição. A data e o horário passam por validações para impedir registros futuros, pois o sistema foi tratado como prontuário eletrônico, ou seja, um registro de atendimentos já realizados, e não como sistema de agendamento.

A função `criarConsultaComDados` foi usada para criar consultas já preenchidas, principalmente durante o carregamento dos dados salvos em arquivo. A função `inserirConsultaInicio` insere a consulta no início da lista encadeada. Já `excluirConsultaPorCodigo` remove uma consulta a partir de seu código, ajustando os ponteiros da lista e liberando a memória com `free`.

As funções `listarConsultas` e `contarConsultas` foram utilizadas para exibir e contabilizar os atendimentos registrados. A função `salvarConsultasArquivo` grava as consultas no arquivo `consultas.txt`, mantendo uma associação com o CPF do paciente para que, ao abrir o sistema novamente, as consultas possam ser vinculadas ao paciente correto.

2.5 Modulo de validação

O modulo de validação foi criado para garantir que o usuário informe dados coerentes com o tipo esperado. Em vez de utilizar `scanf` diretamente em todas as partes do programa, foram criadas funções específicas para leitura segura. A função `lerInteiro` usa `fgets` e `strtol` para aceitar apenas entradas inteiras, impedindo que textos ou caracteres indevidos sejam interpretados como números validos.

A função `lerInteiroPositivo` reaproveita `lerInteiro`, mas exige que o valor seja maior que zero. Ela foi usada em situações como a exclusão de consulta por código. A função `lerInteiroIntervalo` também reaproveita `lerInteiro`, mas restringe a resposta a um intervalo, como ocorre no menu principal, que aceita apenas opções de 1 a 8.

A função `lerTextoValidado` centraliza a leitura de textos. Ela recebe o tipo de validação desejado, como nome, CPF, telefone, data, hora ou texto livre. Essa função também permite cancelar uma operação digitando 0, além de impedir campo vazio, excesso de caracteres e o caractere ponto e vírgula, que poderia prejudicar a estrutura dos arquivos.

Para nomes, foi usada a validação `somenteLetrasEspacos`, que aceita letras e espaços. Para telefone, foi usada `somenteDigitos`, restringindo a entrada a números. Para datas, foram utilizadas as funções `dataValida` e `dataNaoFutura`. `dataValida` verifica o formato `dd/mm/aaaa`, a existência do mês, a quantidade correta de dias e anos bissextos. `dataNaoFutura` compara a data informada com a data atual do computador.

Para horários, a função `horaValida` verifica o formato `hh:mm` e impede horarios inexistentes, como `25:80`. A funcao `dataHoraNaoFutura` utiliza a biblioteca `time.h` para comparar data e hora informadas com o momento atual do sistema. Para isso, usa `struct tm`, `mktime`, `time` e `difftime`. Essa verificação impede cadastrar uma consulta em dia ou horário futuro.

Na validação do CPF, a regra adequada e verificar os dígitos verificadores do documento. O CPF possui 11 dígitos: os nove primeiros formam a base, o decimo digito valida os nove primeiros, e o decimo primeiro válida a sequência seguinte. A função `cpfValido` pode ser utilizada para rejeitar CPFs com todos os dígitos iguais e para recalcular os dois dígitos verificadores, aceitando apenas CPFs matematicamente validos conforme a regra brasileira.

2.6 Persistência em arquivos

A persistência foi feita com arquivos texto. A função `salvarDados` abre os arquivos `pacientes.txt` e `consultas.txt` em modo de escrita. O arquivo `pacientes.txt` recebe os dados principais de cada paciente em formato legível, com rótulos como Nome, CPF, Data de nascimento e Telefone. O arquivo `consultas.txt` recebe as consultas, também em formato detalhado, associando cada consulta ao CPF do paciente.

A função `carregarDados` realiza o processo inverso. Ela abre `pacientes.txt`, identifica linhas iniciadas por Paciente, aloca memória para cada registro e copia os valores dos rótulos para os campos da struct. Depois abre `consultas.txt`, identifica o CPF associado a cada consulta, cria a

consulta com `criarConsultaComDados` e a insere na lista de consultas do paciente correspondente.

Foram usadas funções auxiliares como `removerQuebraLinhaArquivo` e `copiarValorRotulo`. A primeira remove caracteres de quebra de linha lidos do arquivo. A segunda pega apenas o valor depois de um rotulo, por exemplo, transformando a linha `Nome: Max` apenas no valor `Max`. Isso permitiu usar arquivos mais compreensíveis para leitura humana, sem perder a capacidade de carregar os dados novamente no programa.

2.7 Bibliotecas utilizadas

A biblioteca `stdio.h` foi usada para entrada e saída de dados, incluindo `printf`, `fgets`, `fopen`, `fprintf` e `fclose`. A biblioteca `stdlib.h` foi usada para alocação dinâmica com `malloc`, liberação de memória com `free` e conversão de textos para inteiros com `strtol`. A biblioteca `string.h` foi utilizada para manipulação de strings, como `strcmp`, `strcpy`, `strncpy`, `strlen`, `strchr`, `strncmp` e `strcspn`.

A biblioteca `ctype.h` foi usada nas validações de caracteres, permitindo verificar se um caractere é dígito, letra ou espaço. Por fim, `time.h` foi usada para obter a data e hora atuais e comparar com as datas informadas pelo usuário, principalmente na regra que bloqueia datas e horários futuros para consultas.

3. Conclusão

O desenvolvimento do sistema permitiu aplicar conceitos importantes de estrutura de dados e programação modular em C. A separação em TAD Paciente e TAD Consulta tornou o código mais organizado e facilitou a divisão de responsabilidades. O uso de listas encadeadas foi adequado porque o sistema trabalha com quantidades variáveis de pacientes e consultas, permitindo inserir novos registros dinamicamente.

As funções de validação aumentaram a confiabilidade do sistema, evitando entradas incorretas em campos numéricos, textuais, datas, horários e CPF. A utilização de arquivos texto garantiu a persistência dos dados, permitindo que os pacientes e consultas fossem recuperados mesmo depois do encerramento do programa.

Portanto, o trabalho atendeu ao objetivo de construir um prontuário eletrônico funcional, organizado em TADs, com uso de ponteiros, alocação dinâmica, listas encadeadas, validações e arquivos. Além disso, a escolha das funções e estruturas utilizadas foi feita de acordo com a necessidade do problema, priorizando organização, segurança na entrada de dados e facilidade de manutenção.